

Universidade do Minho
Escola de Engenharia

Cálculo de Programas

Trabalho Prático (2025/26)

Lic. em Ciências da Computação
Lic. em Engenharia Informática

Grupo G17

a107293 Hugo Ferreira Soares
a106902 Francisco Ribeiro Martins

Preâmbulo

Em [Cálculo de Programas](#) pretende-se ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em [Haskell](#) (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em [Haskell](#). Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

Antes de abordarem os problemas propostos no trabalho, os grupos devem ler com atenção o anexo [A](#) onde encontrarão as instruções relativas ao *software* a instalar, etc.

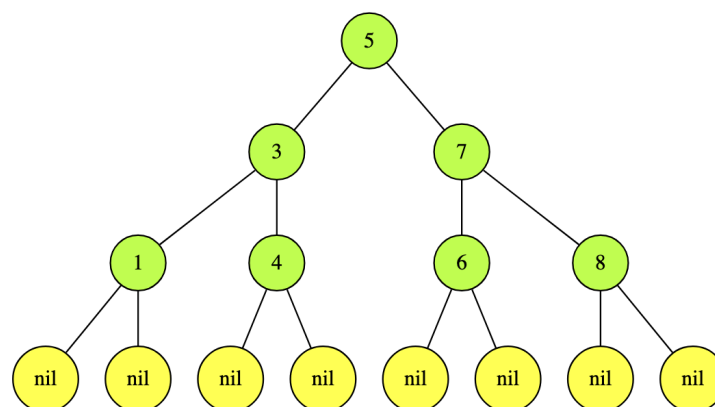
Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Avaliação. Faz parte da avaliação do trabalho a sua defesa por parte dos elementos de cada grupo. Estes devem estar preparados para responder a perguntas sobre *qualquer* dos problemas deste enunciado. A prestação *individual* de cada aluno nessa defesa oral será uma componente importante e diferenciadora da avaliação.

Problema 1

Uma serialização (ou travessia) de uma árvore é uma sua representação sob a forma de uma lista. Na biblioteca *BTree* encontram-se as funções de serialização *inordt*, *preordt* e *postordt*, que fazem as travessias *in-order*, *pre-order* e *post-order*, respectivamente. Todas essas travessias são catamorfismos que percorrem a árvore argumento em regime *depth-first*.

Pretende-se agora uma função *bfordr* que faça a travessia em regime *breadth-first*, isto é, por níveis. Por exemplo, para a árvore t_1 dada em anexo e mostrada na figura a seguir,



a função deverá dar a lista

[5, 3, 7, 1, 4, 6, 8]

em que se vê como os níveis 5, depois 3, 7 e finalmente 1, 4, 6, 8 foram percorridos.

Pretendemos propor duas versões dessa função:

1. Uma delas envolve um catamorfismo de *BTrees*:

$$\begin{aligned} \text{bfsLevels} &:: \text{BTree } a \rightarrow [a] \\ \text{bfsLevels} &= \text{concat} \cdot \text{levels} \end{aligned}$$

Complete a definição desse catamorfismo:

$$\begin{aligned} \text{levels} &:: \text{BTree } a \rightarrow [[a]] \\ \text{levels} &= \llbracket g\text{levels} \rrbracket \end{aligned}$$

2. A segunda proposta,

$$\text{bft} :: \text{BTree } a \rightarrow [a]$$

deverá basear-se num anamorfismo de listas.

Sugestão: estudar o artigo [2] cujo PDF está incluído no material deste trabalho. Quando fizer testes ao seu código pode, se desejar, usar funções disponíveis na biblioteca *Exp* para visualizar as árvores em GraphViz (formato .dot).

Justifique devidamente a sua resolução, que deverá vir acompanhada de diagramas explicativos. Como já se disse, valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Problema 2

Considere a seguinte função em Haskell:

```
f x = wrapper · worker where
  wrapper = head
  worker 0 = start x
  worker (n + 1) = loop x (worker n)
  loop x [s, h, k, j, m] =
    [h / k + s, x ↑ 2 * h, k * j, j + m, m + 8]
  start x = [x, x ↑ 3, 6, 20, 22]
```

Pode-se provar pela lei de recursividade mútua que $f\ x\ n$ calcula o seno hiperbólico de x , $\sinh x$, para n aproximações da sua série de Taylor. Faça a derivação da função dada a partir da referida série de Taylor, apresentando todos os cálculos justificativos, tal como se faz para outras funções no capítulo respectivo do texto base desta UC [3].

Problema 3

Quem em Braga observar, ao fim da tarde, o tráfego onde a Avenida Clairmont Fernand se junta à N101, aproximadamente na coordenada [41°33'46.8"N 8°24'32.4"W](#) — ver as setas da figura que se segue — reparará nas sequências imparáveis (infinitas!) de veículos provenientes dessas vias de circulação.

Mas também irá observar um comportamento interessante por parte dos condutores desses veículos: por regra, *cada carro numa via deixa passar, à sua frente, exactamente outro carro da outra via*.



Este comportamento *civilizado* chama-se *fair-merge* (ou *fair-interleaving*) de duas sequências infinitas, também designadas *streams* em ciência da computação. Seja dado o tipo dessas sequências em Haskell,

data *Stream* *a* = *Cons* (*a*, *Stream* *a*) **deriving** *Show*

para o qual se define também:

out (*Cons* (*x*, *xs*)) = (*x*, *xs*)

O referido comportamento civilizado pode definir-se, em Haskell, da forma seguinte:¹

```
fair_merge :: (Stream a, Stream a) + (Stream a, Stream a) → Stream a
fair_merge = [h, k] where
  h (Cons (x, xs), y) = Cons (x, k (xs, y))
  k (x, Cons (y, ys)) = Cons (y, h (x, ys))
```

Defina *fair_merge* como um **anamorfismo** de *Streams*, usando o combinador

$\llbracket g \rrbracket = \text{Cons} \cdot (\text{id} \times \llbracket g \rrbracket) \cdot g$

e a seguinte estratégia:

- Derivar a lei **dual** da recursividade mútua,

$$[f, g] = \llbracket [h, k] \rrbracket \equiv \begin{cases} \text{out} \cdot f = F [f, g] \cdot h \\ \text{out} \cdot g = F [f, g] \cdot k \end{cases} \quad (1)$$

tal como se fez, nas aulas, para a que está no formulário.

- Usar (1) na resolução do problema proposto.

Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Problema 4

Como se sabe, é possível pensarmos em catamorfismos, anamorfismos etc *probabilísticos*, quer dizer, programas recursivos que dão distribuições como resultados. Por exemplo, podemos pensar num combinador

pcataList :: (() + (*a*, *b*) → *Dist* *b*) → [*a*] → *Dist* *b*

¹ O facto das sequências serem infinitas não nos deve preocupar, pois em Haskell isso é lidado de forma transparente por [lazy evaluation](#).

que é muito parecido com

$$(\cdot) :: () \rightarrow (a, b) \rightarrow b \rightarrow [a] \rightarrow b$$

da biblioteca [List](#). A principal diferença é que o gene de *pcataList* é uma função probabilística.

Como exemplo de utilização, recorde-se que $(\text{zero}, \text{add})$ soma todos os elementos da lista argumento, por exemplo:

$$(\text{zero}, \text{add}) [20, 10, 5] = 35.$$

Considere-se agora a função *padd* (adição probabilística) que, com probabilidade 90% soma dois números e com probabilidade 10% os subtrai:

$$\text{padd}(a, b) = D[(a + b, 0.9), (a - b, 0.1)]$$

Se se correr

$$d4 = \text{pcataList} [\text{pzero}, \text{padd}] [20, 10, 5] \text{ where } \text{pzero} = \text{return} \cdot \text{zero}$$

obter-se-á:

```
35  81.0%
25   9.0%
 5   9.0%
15   1.0%
```

Com base neste exemplo, resolva o seguinte

Problema: Uma unidade militar pretende enviar uma mensagem urgente a outra, mas tem o aparelho de telegrafia meio avariado. Por experiência, o telegrafista sabe que a probabilidade de uma palavra se perder (não ser transmitida) é 5%; e que, no final de cada mensagem, o aparelho envia o código "stop", mas (por estar meio avariado), falha 10% das vezes.

Qual a probabilidade de a palavra "atacar" da mensagem

`words "Vamos atacar hoje"`

se perder, isto é, o resultado da transmissão ser ["Vamos", "hoje", "stop"]? E a de seguirem todas as palavras, mas faltar o "stop" no fim? E a da transmissão ser perfeita?

Responda a estas perguntas encontrando *gene* tal que

`transmitir = pcataList gene`

descreve o comportamento do aparelho. Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Anexos

A Natureza do trabalho a realizar

Este trabalho teórico-prático deve ser realizado por grupos de 3 alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na [página da disciplina](#) na internet.

Recomenda-se uma abordagem participativa dos membros do grupo em **todos** os exercícios do trabalho, para assim poderem responder a qualquer questão colocada na *defesa oral* do relatório.

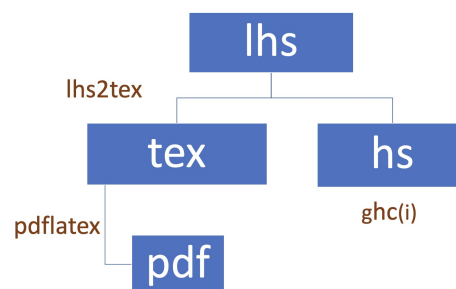
Para cumprir de forma integrada os objectivos do trabalho vamos recorrer a uma técnica de programação dita “literária” [1], cujo princípio base é o seguinte:

Um programa e a sua documentação devem coincidir.

Por outras palavras, o **código fonte** e a **documentação** de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2526t.pdf` que está a ler é já um exemplo de **programação literária**: foi gerado a partir do texto fonte `cp2526t.lhs`¹ que encontrará no **material pedagógico** desta disciplina descompactando o ficheiro `cp2526t.zip`.

Como se mostra no esquema abaixo, de um único ficheiro (*lhs*) gera-se um PDF ou faz-se a interpretação do código **Haskell** que ele inclui:



Vê-se assim que, para além do **GHCI**, serão necessários os executáveis **pdflatex** e **lhs2TeX**. Para facilitar a instalação e evitar problemas de versões e conflitos com sistemas operativos, é recomendado o uso do **Docker** tal como a seguir se descreve.

B Docker

Recomenda-se o uso do **container** cuja imagem é gerada pelo **Docker** a partir do ficheiro `Dockerfile` que se encontra na diretoria que resulta de descompactar `cp2526t.zip`. Este **container** deverá ser usado na execução do **GHCI** e dos comandos relativos ao **LaTeX**. (Ver também a `Makefile` que é disponibilizada.)

Após **instalar o Docker** e descarregar o referido zip com o código fonte do trabalho, basta executar os seguintes comandos:

```
$ docker build -t cp2526t .  
$ docker run -v ${PWD}:/cp2526t -it cp2526t
```

NB: O objetivo é que o container seja usado *apenas* para executar o **GHCI** e os comandos relativos ao **LaTeX**. Deste modo, é criado um *volume* (cf. a opção `-v ${PWD}:/cp2526t`) que permite que a diretoria em que se encontra na sua máquina local e a diretoria `/cp2526t` no **container** sejam partilhadas.

Pretende-se então que visualize/edite os ficheiros na sua máquina local e que os compile no **container**, executando:

¹ O sufixo ‘lhs’ quer dizer *literate Haskell*.

```
$ lhs2TeX cp2526t.lhs > cp2526t.tex
$ pdflatex cp2526t
```

[lhs2TeX](#) é o pre-processador que faz “pretty printing” de código Haskell em [L^AT_EX](#) e que faz parte já do [container](#). Alternativamente, basta executar

```
$ make
```

para obter o mesmo efeito que acima.

Por outro lado, o mesmo ficheiro `cp2526t.lhs` é executável e contém o “kit” básico, escrito em [Haskell](#), para realizar o trabalho. Basta executar

```
$ ghci cp2526t.lhs
```

Abra o ficheiro `cp2526t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo [GHCi](#) para ser executado.

C Em que consiste o TP

Em que consiste, então, o *relatório* a que se referiu acima? É a edição do texto que está a ser lido, preenchendo o anexo [G](#) com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com [BibT_EX](#)) e o índice remissivo (com [makeindex](#)),

```
$ bibtex cp2526t.aux
$ makeindex cp2526t.idx
```

e recompilar o texto como acima se indicou. (Como já se disse, pode fazê-lo correndo simplesmente `make` no [container](#).)

No anexo [F](#) disponibiliza-se algum código [Haskell](#) relativo aos problemas que são colocados. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

Deve ser feito uso da [programação literária](#) para documentar bem o código que se desenvolver, em particular fazendo diagramas explicativos do que foi feito e tal como se explica no anexo [D](#) que se segue.

D Como exprimir cálculos e diagramas em L^AT_EX/lhs2TeX

Como primeiro exemplo, estudar o texto fonte ([lhs](#)) do que está a ler¹ onde se obtém o efeito seguinte:²

$$\begin{aligned} id &= \langle f, g \rangle \\ \equiv \quad &\{ \text{universal property} \} \end{aligned}$$

¹ Procure e.g. por "sec:diagramas".

² Exemplos tirados de [3].

$$\begin{aligned}
& \begin{cases} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{cases} \\
\equiv & \quad \{ \text{identity} \} \\
& \begin{cases} \pi_1 = f \\ \pi_2 = g \end{cases} \\
& \square
\end{aligned}$$

Os diagramas podem ser produzidos recorrendo à *package* [xymatrix](#), por exemplo:

$$\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\
\downarrow \langle g \rangle & & \downarrow id + \langle g \rangle \\
B & \xleftarrow{g} & 1 + B
\end{array}$$

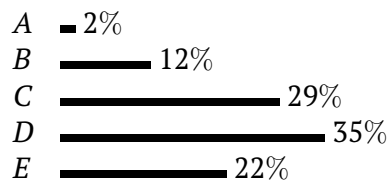
E O mónade das distribuições probabilísticas

Mónades são functores com propriedades adicionais que nos permitem obter efeitos especiais em programação. Por exemplo, a biblioteca [Probability](#) oferece um mónade para abordar problemas de probabilidades. Nesta biblioteca, o conceito de distribuição estatística é captado pelo tipo

$$\text{newtype Dist } a = D \{ unD :: [(a, ProbRep)] \} \quad (2)$$

em que *ProbRep* é um real de 0 a 1, equivalente a uma escala de 0 a 100%.

Cada par (a, p) numa distribuição $d :: \text{Dist } a$ indica que a probabilidade de a é p , devendo ser garantida a propriedade de que todas as probabilidades de d somam 100%. Por exemplo, a seguinte distribuição de classificações por escalões de A a E ,



será representada pela distribuição

$$\begin{aligned}
d1 &:: \text{Dist Char} \\
d1 &= D [('A', 0.02), ('B', 0.12), ('C', 0.29), ('D', 0.35), ('E', 0.22)]
\end{aligned}$$

que o [GHCi](#) mostrará assim:

```

'D'  35.0%
'C'  29.0%
'E'  22.0%
'B'  12.0%
'A'   2.0%

```

É possível definir geradores de distribuições, por exemplo distribuições *uniformes*,

$$d2 = \text{uniform} (\text{words "Uma frase de cinco palavras"})$$

isto é

```

"Uma"    20.0%
"cinco"  20.0%
"de"     20.0%
"frase"  20.0%
"palavras" 20.0%

```

distribuição *normais*, eg.

```
d3 = normal [10..20]
```

etc.¹ Dist forma um **mónade** cuja unidade é $\text{return } a = D [(a, 1)]$ e cuja composição de Kleisli é (simplificando a notação)

$$(f \bullet g) a = [(y, q * p) \mid (x, p) \leftarrow g a, (y, q) \leftarrow f x]$$

em que $g : A \rightarrow \text{Dist } B$ e $f : B \rightarrow \text{Dist } C$ são funções **monádicas** que representam *computações probabilísticas*.

Este mónade é adequado à resolução de problemas de *probabilidades e estatística* usando programação funcional, de forma elegante e como caso particular da programação monádica.

F Código fornecido

Problema 1

Árvores exemplo:

```

t1 :: BTree Int
t1 = Node (5, (Node (3, (Node (1, (Empty, Empty)), Node (4, (Empty, Empty)))),
  Node (7, (Node (6, (Empty, Empty)), Node (8, (Empty, Empty)))))
t2 :: BTree Int
t2 =
  node 1
    (node 2 (node 4 Empty Empty) (node 5 Empty Empty))
    (node 3 (node 6 Empty Empty) (node 7 Empty Empty))
t3 :: BTree Char
t3 =
  node 'A'
    (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
    (node 'E' Empty Empty)
t4 :: BTree Char
t4 =
  node 'A'
    (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
    Empty
t5 :: BTree Int
t5 =
  node 1

```

¹ Para mais detalhes ver o código fonte de [Probability](#), que é uma adaptação da biblioteca [PFP](#) ("Probabilistic Functional Programming"). Para quem quiser saber mais recomenda-se a leitura do artigo [?].

$(node\ 2\ (node\ 4\ Empty\ Empty)\ Empty)$
 $(node\ 3\ Empty\ (node\ 5\ (node\ 6\ Empty\ Empty)\ Empty))$
 $node\ a\ b\ c = Node\ (a,\ (b,\ c))$

G Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o “layout” que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto ao anexo, bem como diagramas e/ou outras funções auxiliares que sejam necessárias.

Importante: Não pode ser alterado o texto deste ficheiro fora deste anexo.

Problema 1

1. Catamorfismo de níveis (*levels*)

Objetivo: Agrupar os elementos de uma árvore binária numa lista de listas, onde cada sublista contém todos os elementos de um determinado nível.

Estratégia: Utilizamos um catamorfismo de árvores binárias, definido como $levels = \langle glevels \rangle$. Um catamorfismo consome a estrutura recursiva da árvore, substituindo cada construtor (*Empty* e *Node*) por operações definidas no gene.

Definição do Gene *glevels*:

O gene processa os dois casos do tipo *BTree a*:

- **Caso base (*Empty*):** Uma árvore vazia não tem níveis, logo retornamos a lista vazia $[]$.
- **Caso recursivo (*Node (a, (l, r))*):** Recebemos o valor da raiz a e as listas de níveis já computadas para as subárvores esquerda l e direita r . A estratégia é:
 1. Criar o nível 0 contendo apenas a raiz: $[a]$
 2. Combinar os níveis restantes de ambas as subárvores usando *zipWithPlus*

Função auxiliar *zipWithPlus*: Esta função funde duas listas de listas, concatenando elemento a elemento. Lida com três casos:

- Se a lista esquerda está vazia, retorna a direita
- Se a lista direita está vazia, retorna a esquerda
- Se ambas têm elementos, concatena as cabeças ($x ++ y$) e aplica recursivamente às caudas

Esta abordagem garante que elementos do mesmo nível (inicialmente em subárvores diferentes) sejam unidos numa única lista.

O diagrama do catamorfismo é o seguinte:

$$\begin{array}{ccc}
 BTree\ a & \xrightarrow{outBTree} & 1 + a \times (BTree\ a \times BTree\ a) \\
 \downarrow levels & & \downarrow id + id \times (levels \times levels) \\
 [[a]] & \xleftarrow{glevels} & 1 + a \times ([[a]] \times [[a]])
 \end{array}$$

`glevels = [[]].comb`

Onde `comb(a,(L,D))=[a]:zipWith (++) L D`.

- `[a]` cria o primeiro nível (a raiz).
- `zipWith (++)` junta o nível n da esquerda com o nível n da direita para formar o novo nível $n+1$.

2. Anamorfismo de travessia (*bft*)

Objetivo: Realizar uma travessia *breadth-first* (por largura) de uma árvore binária, produzindo uma lista com os elementos ordenados por níveis.

Estratégia: Implementamos *bft* como um anamorfismo de listas, seguindo a abordagem de Okasaki baseada numa fila (*queue*). Um anamorfismo *gera* uma estrutura (neste caso, uma lista) a partir de um estado inicial.

Estado: O estado é uma fila de árvores `[BTree a]` que representa as árvores ainda por processar. Inicialmente, a fila contém apenas a árvore original: `[t]`.

Definição do Gene *geneBFT*:

O gene determina o próximo elemento da lista de saída e o novo estado:

- **Caso 1** (fila vazia `[]`): A travessia terminou. Retornamos i_1 `()` para sinalizar o fim da lista.
- **Caso 2** (primeiro elemento é *Empty*): Árvores vazias não contribuem com elementos. Ignoramos e processamos recursivamente o resto da fila.
- **Caso 3** (primeiro elemento é *Node* `(a, (l, r))`):
 1. Extraímos o valor da raiz a como próximo elemento da lista
 2. Adicionamos as subárvores l e r ao **fim** da fila: `ts ++ [l, r]`
 3. Retornamos i_2 `(a, ts ++ [l, r])`

Porque funciona: Ao adicionar subárvores ao fim da fila (comportamento FIFO - *First In, First Out*), garantimos que todas as árvores de um nível são processadas antes das do nível seguinte. Isto assegura a ordem *breadth-first*.

Exemplo: Para a árvore t_1 , a evolução da fila é:

```
[Node 5 ...]      -> 5, fila = [Node 3 ..., Node 7 ...]
[Node 3 ..., ...] -> 3, fila = [Node 7 ..., Node 1 ..., Node 4 ...]
[Node 7 ..., ...] -> 7, fila = [Node 1 ..., Node 4 ..., Node 6 ..., Node 8 ...]
...e assim sucessivamente
```

```
glevels = [[]], λ(a, (ls, rs)) → [a] : zipWithPlus ls rs
where zipWithPlus [] ys = ys
      zipWithPlus xs [] = xs
      zipWithPlus (x : xs) (y : ys) = (x ++ y) : zipWithPlus xs ys
bft t = [(geneBFT)] [t]
where geneBFT [] = i1 ()
      geneBFT (Empty : ts) = geneBFT ts
      geneBFT (Node (a, (l, r)) : ts) = i2 (a, ts ++ [l, r])
```

Problema 2

Resolução Teórica

Objetivo: Calcular o seno hiperbólico de x usando n termos da sua série de Taylor, de forma eficiente.

Série de Taylor do $\sinh$:

$$\sinh(x) = \sum_{i=0}^{\infty} a_i \quad \text{onde} \quad a_i = \frac{x^{2i+1}}{(2i+1)!}$$

Os primeiros termos são:

$$\begin{aligned} a_0 &= \frac{x^1}{1!} = x \\ a_1 &= \frac{x^3}{3!} = \frac{x^3}{6} \\ a_2 &= \frac{x^5}{5!} = \frac{x^5}{120} \\ a_3 &= \frac{x^7}{7!} = \frac{x^7}{5040} \end{aligned}$$

Problema: Calcular cada termo independentemente seria muito ineficiente (fatoriais e potências crescem rapidamente).

Solução: Usar uma relação de recorrência. Observando a relação entre termos consecutivos:

$$\frac{a_{i+1}}{a_i} = \frac{x^{2(i+1)+1}}{(2(i+1)+1)!} \cdot \frac{(2i+1)!}{x^{2i+1}} = \frac{x^2}{(2i+2)(2i+3)}$$

Portanto: $a_{i+1} = a_i \cdot \frac{x^2}{(2i+2)(2i+3)}$

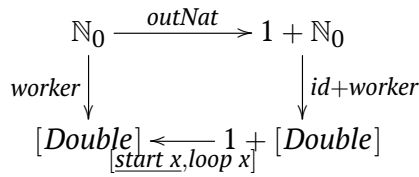
Estratégia de implementação: Mantemos acumuladores para:

- s : Soma acumulada dos termos já calculados
- h : Numerador do próximo termo (potência de x)
- k : Denominador do próximo termo (fatorial)
- j : Multiplicador para atualizar o fatorial: $(2i+2)(2i+3)$
- m : Incremento para j (aumenta 8 a cada iteração)

Derivação das relações:

$$\begin{aligned} h_i &= x^{2i+1} \Rightarrow h_{i+1} = h_i \cdot x^2 \\ k_i &= (2i+1)! \Rightarrow k_{i+1} = k_i \cdot (2i+2)(2i+3) \\ j_i &= (2i+2)(2i+3) = 4i^2 + 10i + 6 \\ j_{i+1} - j_i &= 4(i+1)^2 + 10(i+1) + 6 - (4i^2 + 10i + 6) = 8i + 14 \\ (\text{simplificando}) \quad m_i &= 8i + 14 \quad (\text{começa em 22 para } i = 1) \end{aligned}$$

O catamorfismo de números naturais (*worker*) evolui conforme o seguinte diagrama:



Análise das Componentes do Estado:

Para $n = 0$, o estado inicial é $\text{start } x = [x, x \uparrow 3, 6, 20, 22]$, correspondendo a $[s, h, k, j, m]$:

- $s = x$: Soma já contém $a_0 = x$ (primeiro termo da série)
- $h = x^3$: Numerador pronto para calcular $a_1 = \frac{x^3}{6}$
- $k = 6 = 3!$: Denominador para a_1
- $j = 20 = 4 \cdot 5$: Multiplicador para obter $5! = 3! \cdot 4 \cdot 5$ (necessário para a_2)
- $m = 22$: Incremento para j (de 20 para $6 \cdot 7 = 42$ na próxima iteração)

Iteração do loop: A cada chamada, $\text{loop } x [s, h, k, j, m]$ calcula:

- Novo s : soma acumulada mais o termo atual $\frac{h}{k}$
- Novo h : multiplica por x^2 para a próxima potência
- Novo k : multiplica por j para o próximo fatorial
- Novo j : incrementa por m para manter a sequência correta
- Novo m : incrementa 8 para o próximo salto

Exemplo de execução para $f \ 1.0 \ 2$:

Iteracao 0: $[1.0, 1.0, 6, 20, 22]$

Iteracao 1: $[1.0 + 1.0/6, 1.0, 120, 42, 30] = [1.1667, 1.0, 120, 42, 30]$

Iteracao 2: $[1.1667 + 1.0/120, 1.0, 5040, 72, 38] = [1.175, 1.0, 5040, 72, 38]$

Resultado: $\text{head } [1.175, \dots] = 1.175$, próximo de $\sinh(1.0) \approx 1.1752$

O *wrapper* (função *head*) extrai o primeiro elemento da lista final, que contém a soma de todos os $n + 1$ termos da série.

Implementação

```

f :: Double → Int → Double
f x n = wrapper (worker x n)
  where
    wrapper = head
    worker x = ([start x, loop x])
    start x = [x, x ↑ 3, 6, 20, 22]
    loop x [s, h, k, j, m] = [s + h / k, x ↑ 2 * h, k * j, j + m, m + 8]

sumList :: Num a ⇒ [a] → a
sumList = ([0, (+)])

productList :: Num a ⇒ [a] → a
productList = ([1, (*)])

```

$$\begin{aligned} \text{lengthList} &:: [a] \rightarrow \text{Int} \\ \text{lengthList} &= \llbracket [0, \text{succ} \cdot \pi_2] \rrbracket \end{aligned}$$

Problema 3

Objetivo: Implementar uma função *fairMerge* que intercala dois streams (listas infinitas) de forma alternada, usando um anamorfismo.

Contexto: O problema é motivado pela situação real de tráfego onde carros de duas vias se intercalam de forma *justa* (cada carro deixa passar um da outra via).

Lei de Fokkinga Dual

Propósito: A Lei de Fokkinga Dual é a versão para anamorfismos da lei de recursividade mútua. Permite fundir dois anamorfismos mutuamente recursivos num único anamorfismo, usando o combinador $[\cdot, \cdot]$.

Enunciado da lei:

$$\llbracket [h], [k] \rrbracket = \llbracket [f, g] \rrbracket$$

sob as condições:

$$\begin{cases} \text{out} \cdot \llbracket [h] \rrbracket = F \llbracket [h], [k] \rrbracket \cdot f \\ \text{out} \cdot \llbracket [k] \rrbracket = F \llbracket [h], [k] \rrbracket \cdot g \end{cases}$$

Interpretação: Dois processos geradores h e k que se referenciam mutuamente podem ser fundidos num único gerador que escolhe (via $[\cdot, \cdot]$) qual comportamento seguir em cada passo.

Para Streams, onde $F X = A \times X$, isto significa que podemos ter dois geradores que produzem streams alternando entre si, e fundi-los num único anamorfismo.

Derivação da Lei de Fokkinga Dual

A lei dual da recursividade mútua (Fokkinga Dual) fundamenta-se na propriedade universal do produto para anamorfismos. Pretendemos provar que:

$$\llbracket [f], [g] \rrbracket = \llbracket [h, k] \rrbracket \tag{3}$$

Assumindo a estrutura de Streams, onde o functor é $F X = A \times X$, a prova desenrola-se da seguinte forma:

$$\begin{aligned} &\begin{cases} \text{out} \cdot f = F [f, g] \cdot h \\ \text{out} \cdot g = F [f, g] \cdot k \end{cases} \\ \equiv &\quad \{ \text{Isomorfismo out/in; Exponencialização} \} \\ &\begin{cases} \text{out} \cdot f = (id \times [f, g]) \cdot h \\ \text{out} \cdot g = (id \times [f, g]) \cdot k \end{cases} \\ \equiv &\quad \{ \text{Propriedade Universal do Coproduto (Either)} \} \\ &[\text{out} \cdot f, \text{out} \cdot g] = (id \times [f, g]) \cdot [h, k] \\ \equiv &\quad \{ \text{Lei da Fusão do Either: } [f \cdot i, g \cdot j] = f + g \} \\ &\text{out} \cdot [f, g] = (id \times [f, g]) \cdot [h, k] \end{aligned}$$

$$\equiv \{ \text{Definição de Anamorfismo (Propriedade Universal)} \}$$

$$[f, g] = \llbracket [h, k] \rrbracket$$

$$\square$$

Esta derivação demonstra que o par de processos independentes f e g pode ser fundido num único anamorfismo cujo gene é a escolha $([\cdot, \cdot])$ entre os dois comportamentos originais. No caso do *fairMerge*, isto permite que o estado alterne entre consumir a "Estrada 1" ou a "Estrada 2".

Diagrama do Anamorfismo

Estratégia para *fairMerge*:

Em vez de definir *fairMerge* por recursividade mútua (como h e k no código original), aplicamos a Lei de Fokkinga Dual para obter um único anamorfismo.

O *fairMerge* alterna entre duas streams. O estado do anamorfismo é o par de streams $(\text{Stream } a, \text{Stream } a)$ e, em cada passo, o sistema:

1. Retira o elemento da cabeça da primeira stream
2. Troca a ordem das streams no estado para que na próxima iteração processe a outra

$$\begin{array}{ccc} \text{Stream } a & \xrightarrow{\text{out}} & a \times \text{Stream } a \\ \text{fairMerge} \uparrow & & \uparrow \text{id} \times \text{fairMerge} \\ (\text{Stream } a, \text{Stream } a) & \xrightarrow{\text{gene}} & a \times (\text{Stream } a, \text{Stream } a) \end{array}$$

Análise do Gene: O gene do *fairMerge* recebe o estado $(s1, s2)$ (par de streams) e:

1. Usa *out* $s1$ para decompor a primeira stream em cabeça h e cauda t
2. Retorna $(h, (s2, t))$, onde:
 - h é o próximo elemento do stream resultante
 - $(s2, t)$ é o novo estado, com as streams trocadas

Porque funciona: A inversão de $s2$ com t no novo estado $(s2, t)$ garante que:

- Na iteração atual, consumimos de $s1$ ("Estrada 1")
- Na próxima iteração, o estado é $(s2, t)$, logo consumiremos de $s2$ ("Estrada 2")
- Na iteração seguinte, o estado será $(t, s2')$, voltando a consumir da original $s1$

Esta troca sistemática produz a alternância perfeita: $s1_0, s2_0, s1_1, s2_1, s1_2, s2_2, \dots$

Nota sobre *fairMergeM*: Esta variante usa um parâmetro adicional p (probabilidade) para decidir de qual stream consumir, permitindo modelar situações onde a alternância não é perfeita (e.g., tráfego com diferentes prioridades).

Implementação

```
fairMerge :: (Stream a, Stream a) → Stream a
fairMerge = \llbracket gene \rrbracket
where
```

```

gene (s1, s2) = let (h, t) = out s1
in (h, (s2, t))
fairMergeM :: (Double, (Stream a, Stream a)) → Stream a
fairMergeM = [(geneM)]
where
geneM (p, (s1, s2)) =
  let r = 0.5
  in if p > r
    then let (h, t) = out s1 in (h, (p, (s2, t)))
    else let (h, t) = out s2 in (h, (p, (s1, t)))

```

Problema 4

Objetivo: Modelar o comportamento probabilístico de um aparelho de telegrafia avariado que pode perder palavras durante a transmissão.

Resolução Teórica

Modelo: Usamos um catamorfismo probabilístico *pcataList gene* que, em vez de devolver um resultado determinístico, devolve uma distribuição de probabilidades sobre possíveis resultados.

Tipo: *transmitir* :: [String] → Dist [String]

O gene processa a mensagem (lista de palavras) recursivamente:

- **Caso base** (i_1 ()), lista vazia):

Chegámos ao fim da mensagem. O aparelho tenta enviar "stop":

- Com probabilidade 90%: sucesso → resultado é ["stop"]
- Com probabilidade 10%: falha → resultado é [] (sem stop)

Código: $D [(["stop"], 0.9), ([], 0.1)]$

- **Caso recursivo** (i_2 (w, d)):

Recebemos a palavra atual *w* e a distribuição *d* de possíveis continuações (resultado do processamento do resto da lista).

Para cada possível continuação *msg* em *d*, a palavra *w*:

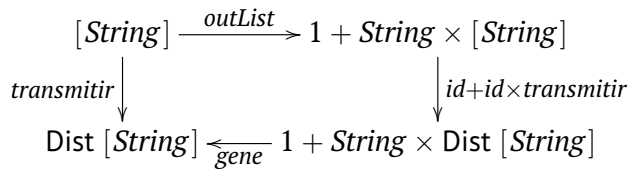
- Com probabilidade 95%: é transmitida → resultado é *w* : *msg*
- Com probabilidade 5%: perde-se → resultado é *msg* (sem *w*)

Usamos notação monádica: **do** { *msg* ← *d*; $D [(w : msg, 0.95), (msg, 0.05)]$ }

Isto combina as probabilidades: se *msg* tem probabilidade *p* em *d*, então:

- *w* : *msg* terá probabilidade $p \times 0.95$
- *msg* (sem *w*) terá probabilidade $p \times 0.05$

Diagrama do Catamorfismo



Cálculo das Probabilidades

Para a mensagem *words* "Vamos atacar hoje" = ["Vamos", "atacar", "hoje"]:

- **Transmissão perfeita:** Manter as 3 palavras ($0.95^3 = 0.857375$) e enviar "stop" (0.9):

$$P(\text{perfeita}) = 0.857375 \times 0.9 = 0.7716 \text{ (77.16\%)}$$

- **Perder "atacar":** Manter "Vamos" (0.95), perder "atacar" (0.05), manter "hoje" (0.95), enviar "stop" (0.9):

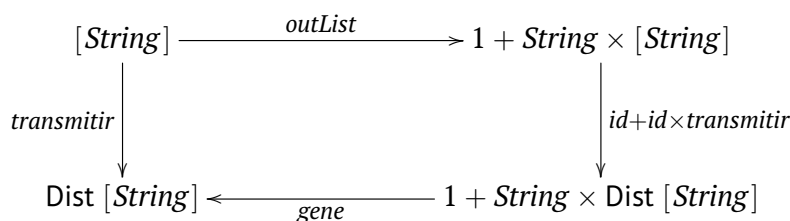
$$P(\text{perder "atacar"}) = 0.95 \times 0.05 \times 0.95 \times 0.9 = 0.0406 \text{ (4.06\%)}$$

- **Todas as palavras mas sem "stop":** Manter as 3 palavras (0.857375) e falhar "stop" (0.1):

$$P(\text{sem stop}) = 0.857375 \times 0.1 = 0.0857 \text{ (8.57\%)}$$

Diagrama do Catamorfismo Probabilístico

O processo de transmissão é modelado por um catamorfismo probabilístico sobre listas. O diagrama ilustra como a estrutura recursiva da mensagem é consumida para gerar uma distribuição estatística de possíveis resultados:



A utilização deste diagrama justifica a composição monádica das probabilidades, onde o gene lida com a incerteza de cada palavra individualmente, propagando o efeito para o resultado global da mensagem.

Resposta às questões:

Para a mensagem ["Vamos", "atacar", "hoje"]:

1. Probabilidade de perder "atacar": 4.06%

Caminho: Vamos (mantida) → atacar (perdida) → hoje (mantida) → stop (enviado)

$$P = 0.95 \times 0.05 \times 0.95 \times 0.9 = 0.0406$$

2. Probabilidade de transmitir todas mas sem "stop": 8.57%

Caminho: Vamos (mantida) → atacar (mantida) → hoje (mantida) → stop (falhou)

$$P = 0.95^3 \times 0.1 = 0.857375 \times 0.1 = 0.0857$$

3. Probabilidade de transmissão perfeita: 77.16%

Caminho: Vamos (mantida) → atacar (mantida) → hoje (mantida) → stop (enviado)

$$P = 0.95^3 \times 0.9 = 0.857375 \times 0.9 = 0.7716$$

Verificação: A soma de todas as probabilidades possíveis (há $2^4 = 16$ combinações) deve ser 100%. Os três casos acima são apenas alguns dos possíveis resultados.

Implementação

```
gene :: () + (String, Dist [String]) → Dist [String]
gene = [pzero, padd]
  where
    pzero () = D [(["stop"], 0.9), ([], 0.1)]
    padd (w, d) = do { msg ← d; D [(w : msg, 0.95), (msg, 0.05)] }
```

Index

$\LaTeX$, [5](#), [6](#)

bibtex, [6](#)

lhs2TeX, [5](#), [6](#)

makeindex, [6](#)

pdflatex, [5](#)

xymatrix, [7](#)

Combinador “pointfree”

ana, [3](#)

cata

 Naturais, [7](#)

either, [3](#), [4](#)

split, [6](#)

Cálculo de Programas, [1](#), [4](#)

 Material Pedagógico, [5](#)

 List.hs, [4](#)

Docker, [5](#)

 container, [5](#), [6](#)

Functor, [3](#), [7](#), [8](#)

Função

π_1 , [7](#)

π_2 , [7](#)

Haskell, [1](#), [5](#), [6](#)

 Biblioteca

 PFP, [8](#)

 Probability, [7](#), [8](#)

 interpretador

 GHCi, [5–7](#)

 Lazy evaluation, [3](#)

 Literate Haskell, [5](#)

Números naturais ($\mathbb{N}$), [7](#)

Programação

 literária, [5](#), [6](#)

References

- [1] D.E. Knuth. *Literate Programming*. CSLI Lecture Notes Number 27. Stanford University Center for the Study of Language and Information, Stanford, CA, USA, 1992.
- [2] Chris Okasaki. Breadth-first numbering: lessons from a small exercise in algorithm design. In Martin Odersky and Philip Wadler, editors, *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, pages 131–136. ACM, 2000.
- [3] J.N. Oliveira. Program Design by Calculation, 2024. Draft of textbook in preparation. First version: 1998. Current version: Sep. 2024. Informatics Department, University of Minho ([pdf](#)).